
Calclatorz

**Arithmetic Expression Evaluator in C++
Software Architecture Document**

Version 1.4

Arithmetic Expression Evaluator in C++	Version: 1.4
Software Architecture Document	Date: 2026-04-07

Revision History

Date	Version	Description	Author
2026-03-06	1.0	Drafted introduction	Chase + Group
2026-03-10	1.1	Drafted logical view	Eian + Group
2026-03-24	1.2.1	Drafted architectural representation, goals, and constraints	Jason
2026-03-24	1.2.2	Drafted quality section and added interface mockup	Chase + Group
2026-03-31	1.3	Merged versions, formatted, and polished	Isaiah
2026-04-0	1.4	Minor architecture updates to align with layered style	Isaiah + Group

Arithmetic Expression Evaluator in C++	Version: 1.4
Software Architecture Document	Date: 2026-04-07

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 References	4
1.5 Overview	4
2. Architectural Representation	4
3. Architectural Goals and Constraints	4
4. Use-Case View	5
4.1 Use-Case Realizations	5
5. Logical View	5
5.1 Overview	5
5.2 Architecturally Significant Design Modules or Packages	5
6. Interface Description	6
7. Size and Performance	6
8. Quality	6

Arithmetic Expression Evaluator in C++	Version: 1.4
Software Architecture Document	Date: 2026-04-07

Software Architecture Document

1. Introduction

1.1 Purpose

The Software Architecture Document describes the structure of our program on a fundamental level. It is intended to convey the specific architectural decisions that have been made on the system.

1.2 Scope

This *Software Development Plan* describes the plan to be used by the Arithmetic Expression Evaluator in C++ project, including deployment of the product. The details of the individual iterations will be described in the Iteration Plans.

1.3 Definitions, Acronyms, and Abbreviations

See the Glossary in the *User's Manual*.

1.4 References

Project Description, *Software Requirements Specifications*, *Software Architecture Document*, *Test Case*, and *User's Manual*.

1.5 Overview

This document provides an overview and breakdown of the major internal function of the program as well as how the functions needed for functionality is structured and hierarchically placed within the program.

2. Architectural Representation

The system uses a modular layered architecture, where each module handles a specific step in processing an expression. Each module is independent, improving maintainability and debugging. The main components are as follows:

- User Interface: handles input and displays results/errors
- Error Handler: detects and reports errors with positions
- Tokenizer: converts input into tokens for a more structured evaluation
- Prioritization: applies operator precedence and organizes tokens
- Evaluation: compute the final result
- History: manages cross-session memory

3. Architectural Goals and Constraints

Goals:

- Modularity: separate UI, tokenizer, parser, and evaluator
- Correctness: follow standard arithmetic rules
- Error Handling: detect invalid input and highlight errors
- Usability: clear results and messages

Constraints:

- Implemented in C++
- UI built with Qt
- Supports basic operations
- Handles single-line expressions
- Run efficiently

Arithmetic Expression Evaluator in C++	Version: 1.4
Software Architecture Document	Date: 2026-04-07

4. Use-Case View

Our primary use case will be getting the results of computations used during tabletop role-playing games. This includes dice rolling of standard tabletop roleplaying game dice (having 4, 6, 8, 10, 12, 20, and 100 sides). Alongside computing any kind of arithmetic the user may input.

4.1 Use-Case Realizations

A specific realization would be a user attending of a Dungeons and Dragons meeting and needing to roll two 20-sided dice and use the highest to figure out if they can hit an enemy. Then rolling two 6-sided dice, adding the values together and adding an additional modifier of 4.

Another use case for the software would be if two people wanted to use the software for a virtual game of Yahtzee, where the users would use the dice randomization. The users would roll 5 different 6-sided die and take the sum of those die for points.

5. Logical View

5.1 Overview

Tokenizer, Prioritizer, and Evaluator will have their own module, which will be included in the other files needed for the program to function.

5.2 Architecturally Significant Design Modules or Packages

randomDiceRoll:

Description: Receives input from the user from the interface then performs the random dice roll for example, if the user wanted to roll 2 D8 dice it would perform a function like (D8) + (D8).

arithmeticOperators:

Description: Takes the input from the interface through the tokenizer specifically the arithmetic operators and then performs the operation with the numerical values provided by the user.

Precedence:

Description: Take the current order of the tokens, compute precedence according to precedence token placement into a standard notation.

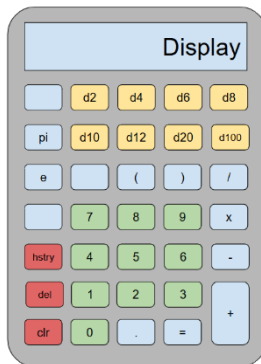
History:

Description: Save past operations into an internal or hidden file.

Arithmetic Expression Evaluator in C++	Version: 1.4
Software Architecture Document	Date: 2026-04-07

6. Interface Description

The interface will be a standard calculator interface based in QT that includes functions, basic arithmetic (multiplication, division, addition, and subtraction), numeral buttons (0-9, decimal), a clear button, and buttons for randomized dice rolls for standard tabletop dice (D20, D12, D10, D8, D6, D4, and percentile dice). Pressing the “clr” button will clear the display of all characters. When pressing the history, a secondary display will be presented with previous equations input into calculator with their answers. Within this interface, there will also be a clear history button that clears all history of previous computations.



7. Size and Performance

Computations will execute in less than one second. We will make the total file space less than a gigabyte.

8. Quality

The calculator will be delivered as a downloadable application that runs entirely offline, improving portability and ensuring it can be used without an internet connection. By avoiding reliance on external services, the system minimizes potential points of failure and increases reliability and availability.

The architecture emphasizes correctness and stability through well-defined computational logic and proper input validation, ensuring accurate and predictable results. Basic error handling (e.g., division by zero) will further enhance system robustness.

From a security and privacy standpoint, the application poses minimal risk. All processing occurs locally, with no data transmission or storage of user information, eliminating concerns about data breaches or unauthorized access. This design significantly reduces the system's attack surface.

The system will follow a modular structure, separating the user interface from the calculation logic. This supports maintainability and allows for future extensions, such as adding advanced functions, without major redesign.